The Journal of Open Source Software

HiGP: A high-performance Python package for Gaussian Processes

Hua Huang [1*], **Tianshi Xu** [2*], **Yuanzhe Xi** [2], and **Edmond Chow** [1¶]

1 School of Computational Science, Engineering, Georgia Institute of Technology, USA **2** Department of Mathematics, Emory University, USA ¶ Corresponding author * These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 04 July 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Gaussian Processes (GPs) (Rasmussen & Williams, 2005) are flexible, nonparametric Bayesian models widely used for regression and classification tasks due to their ability to capture complex data patterns and provide uncertainty quantification. Traditional GP implementations often face challenges in scalability as data sizes increase. To address these challenges, HiGP, a high-performance Python package, is designed for efficient Gaussian Process regression and classification across datasets of varying sizes. HiGP uses iterative solvers to access large GP computations. It implements matrix-vector (MatVec) and matrix-matrix (MatMul) multiplication algorithms tailored to kernel matrices by exploiting their hierarchical low-rank structure (Cai et al., 2023; Huang et al., 2020; Xing & Chow, 2020). To improve the convergence of iterative methods, HiGP also integrates the Adaptive Factorized Nyström (AFN) preconditioner (Zhao et al., 2024) which is designed to be robust across a wide range of kernel hyperparameter values used during hyperparameter optimization. HiGP can be used with PyTorch and other Python packages, allowing it to be incorporated into common machine learning and data analysis workflows.

Gaussian Processes

For training points $\mathbf{X} \in \mathbb{R}^{n\times d}$, a noisy training observation set $\mathbf{y} \in \mathbb{R}^n$, and testing points $\mathbf{X}_* \in \mathbb{R}^{m\times d}$, a standard GP model assumes that the noise-free testing observations $\mathbf{y}_* \in \mathbb{R}^m$ follow the joint distribution:

$$\begin{bmatrix}\mathbf{y}\\ \mathbf{y}_*\end{bmatrix} \sim \mathcal{N}\left(\mathbf{0}, f^2\begin{bmatrix}\kappa(\mathbf{X},\mathbf{X})+s\mathbf{I} & \kappa(\mathbf{X},\mathbf{X}_*)\\ \kappa(\mathbf{X}_*,\mathbf{X}) & \kappa(\mathbf{X}_*,\mathbf{X}_*)\end{bmatrix}\right). \tag{1}$$

Here, $f > 0$ and $s \geq 0$ are real numbers, $\mathbf{I}$ is the identity matrix, $\kappa(\mathbf{u},\mathbf{v}) : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$ is a kernel function, and $\kappa(\mathbf{X},\mathbf{Y})$ is a kernel matrix with the (i,j)-th entry defined as $\kappa(\mathbf{X}_{i,:},\mathbf{Y}_{j,:})$, where $\mathbf{X}_{i,:}$ denotes the i-th row of $\mathbf{X}$. Commonly used kernel functions include the Gaussian kernel (also known as the Radial Basis Function kernel) and Mat'ern kernels. These kernel functions typically depend on one or more kernel parameters. For example, the Gaussian kernel $\kappa(\mathbf{u},\mathbf{v}) = \exp(-\|\mathbf{u}-\mathbf{v}\|^2/(2l^2))$ depends on the parameter $l > 0$, typically known as the length scale.

To find the s, f, and kernel parameter l that best fit the data, an optimization process is generally required to minimize the negative log marginal likelihood:

$$L(\Theta) = \frac{1}{2}\left(\mathbf{y}^\top\widehat{\mathbf{K}}^{-1}\mathbf{y} + \log|\widehat{\mathbf{K}}| + n\log 2\pi\right), \tag{2}$$

where $\widehat{\mathbf{K}}$ denotes the regularized kernel matrix $f^2\kappa(\mathbf{X},\mathbf{X}) + s\mathbf{I}$ and Θ denotes the hyperpara-

meter set (s, f, l). An optimization process usually requires the gradient of $L(\Theta)$:

$$\frac{\partial L}{\partial \theta} = \frac{1}{2}\left(-\mathbf{y}^\top \widehat{\mathbf{K}}^{-1}\frac{\partial \widehat{\mathbf{K}}}{\partial \theta}\widehat{\mathbf{K}}^{-1}\mathbf{y} + \mathrm{tr}\left(\widehat{\mathbf{K}}^{-1}\frac{\partial \widehat{\mathbf{K}}}{\partial \theta}\right)\right), \quad \theta \in \Theta. \tag{3}$$

For small or moderate size datasets, $\widehat{\mathbf{K}}$, $\widehat{\mathbf{K}}^{-1}$, and $\partial\widehat{\mathbf{K}}/\partial\theta$ can be formed explicitly, and Formulas (2) and (3) can be calculated exactly. For large datasets, it is often intractable to populate and store $\widehat{\mathbf{K}}$, $\widehat{\mathbf{K}}^{-1}$, or $\partial\widehat{\mathbf{K}}/\partial\theta$, as these matrices require $\mathcal{O}(n^2)$ space for storage and $\widehat{\mathbf{K}}^{-1}$ requires $\mathcal{O}(n^3)$ arithmetic operations to compute. Instead, using preconditioned iterative methods with preconditioner $M \approx \tilde{K}$ is often a better option (Aune et al., 2014), (Zhang et al., 2024), (Titsias, 2009), (Hensman et al., 2013), (Wilson et al., 2015), (Pleiss et al., 2018), (Wenger et al., 2022), (Chen et al., 2023). In this approach, $\mathbf{K}^{-1}\mathbf{y}$ is approximated via the Preconditioned Conjugate Gradient (PCG) method (Saad, 2003). The trace term $\mathrm{tr}(\widehat{\mathbf{K}}^{-1}\frac{\partial\widehat{\mathbf{K}}}{\partial\theta})$ can be rewritten as

$$\mathrm{tr}\left(\mathbf{M}^{-1}\frac{\partial \mathbf{M}}{\partial \theta}\right) + \mathrm{tr}\left(\widehat{\mathbf{K}}^{-1}\frac{\partial \widehat{\mathbf{K}}}{\partial \theta} - \mathbf{M}^{-1}\frac{\partial \mathbf{M}}{\partial \theta}\right),$$

and the second term can then be estimated by the Hutchinson estimator (Hutchinson, 1989; Meyer et al., 2021):

$$\mathrm{tr}\left(\widehat{\mathbf{K}}^{-1}\frac{\partial \widehat{\mathbf{K}}}{\partial \theta} - \mathbf{M}^{-1}\frac{\partial \mathbf{M}}{\partial \theta}\right) \approx \frac{1}{k}\sum_{i=1}^{k}\mathbf{z}_i^\top\left(\widehat{\mathbf{K}}^{-1}\frac{\partial \widehat{\mathbf{K}}}{\partial \theta} - \mathbf{M}^{-1}\frac{\partial \mathbf{M}}{\partial \theta}\right)\mathbf{z}_i, \tag{4}$$

where $\mathbf{z}_i \sim \mathcal{N}(0, I), i = 1, \cdots, k$ are independent random vectors. To estimate $\log|\hat{\mathbf{K}}|$ in Equation (2), we use stochastic Lanczos quadrature (Ubaru et al., 2017) for $\mathrm{tr}(\log|\hat{\mathbf{K}}|)$. This method needs to sample k_z independent vectors $\mathbf{z}_i \sim \mathcal{N}(0, I), i = 1, \cdots, k_z$ and solve linear systems

$$\widehat{\mathbf{K}}\mathbf{u}_i = \mathbf{z}_i, \quad i = 1, 2, \ldots, k_z. \tag{5}$$

using PCG with preconditioner M. Let T_{z_i} denote the tridiagonal matrix in the Lanczos method on matrix $\mathbf{M}^{-1/2}\widehat{\mathbf{K}}\mathbf{M}^{-1/2}$. Using the identity $\log|\mathbf{K}| = \log|\mathbf{M}| + \log|\mathbf{M}^{-1/2}\widehat{\mathbf{K}}\mathbf{M}^{-1/2}|$, we only apply the Hutchinson estimator to approximate the second term:

$$\log|\mathbf{M}^{-1/2}\widehat{\mathbf{K}}\mathbf{M}^{-1/2}| = \mathrm{tr}\left(\log \mathbf{M}^{-1/2}\widehat{\mathbf{K}}\mathbf{M}^{-1/2}\right) \approx \frac{1}{k_z}\sum_{i=1}^{k_z}\|\mathbf{z}_i\|^2\mathbf{e}_1^\top \log(\mathbf{T}_{\mathbf{z}_i})\mathbf{e}_1, \tag{6}$$

where $\mathbf{e}_1 = [1, 0, 0, \ldots, 0]^\top$. This preconditioned trace estimation approach is particularly efficient when $\log|\mathbf{M}|$ and $\mathrm{tr}\left(\mathbf{M}^{-1}\frac{\partial\mathbf{M}}{\partial\theta}\right)$ can be computed efficiently, leveraging the simpler structure of $\mathbf{M}$ compared to that of $\widehat{\mathbf{K}}$. The AFN preconditioner used in HiGP has a triangular factorization structure, which makes such computation efficient.

Statement of Need

GP research has undergone significant innovations in recent years, including advances in deep Gaussian processes (DGPs) (Bui et al., 2016), (Damianou & Lawrence, 2013), (Jakkala, 2021), (Dutordoir et al., 2021), preconditioned GPs (Wenger et al., 2022) (Wagner et al., 2025), and unbiased GPs (Potapczynski et al., 2021) (Burt et al., 2021). Additionally, there has been a growing focus on improving the accuracy and stability of GP models for large datasets as well as accelerating computations in GP using modern hardware like graphics processing units (GPUs). Multiple GP packages have been released in recent years to address these computational challenges. The GPyTorch package (Gardner et al., 2018) is built on top of PyTorch to leverage GPU computing capabilities. Similarly, GPflow (Matthews et al., 2017;

DRAFT

van der Wilk et al., 2020) leverages another deep learning framework, TensorFlow (Abadi et al., 2015), for GPU acceleration. GPy (GPy, since 2012) is supported by NumPy (Harris et al., 2020) with some GPU support.

The motivation for HiGP's development is to utilize new numerical algorithms and parallel computing techniques to reduce computational complexity and to improve computation efficiency of the iterative methods in GP model training. Compared to existing packages, HiGP has three main advantages and contributions.

Firstly, HiGP addresses the efficiency of MatVec, the most performance-critical operation in iterative methods. Traditional methods populate and store $\mathbf{K}$ and $\partial\widehat{\mathbf{K}}/\partial\theta$ for MatVec, but the $\mathcal{O}(n^2)$ storage and computation costs become prohibitive for large datasets, such as when $n \geq 10,000$. HiGP utilizes two methods to address this issue: the $\mathcal{H}^2$ matrix Hackbusch & Börm (2002) and on-the-fly computation mode. For large 2D or 3D datasets (e.g. spatial data), the dense kernel matrix is compressed into a $\mathcal{H}^2$ matrix in HiGP, resulting in $\mathcal{O}(n)$ storage and computation costs. This approach is also used in the smashGP package (Erlandson et al., 2025). For large high-dimensional datasets, HiGP computes small blocks of the kernel matrix on-the-fly and immediately uses these blocks in MatVec before discarding them instead of storing them in memory. This on-the-fly mode allows HiGP to handle extremely large datasets on a computer with moderate memory size.

Secondly, HiGP adopts a scalable computational approach: iterative solvers with a robust preconditioner. In GP model training, changes in hyperparameters result in variations in the kernel matrix's spectrum. Direct methods are robust against changes in the matrix spectrum, but the $\mathcal{O}(n^3)$ computational costs make them unaffordable for large datasets. Iterative solvers are sensitive to the matrix spectrum and might fail to provide solutions with the desired accuracy. Existing GP packages usually use simple preconditioners, such as a low-rank Nyström approximate factorization of the kernel matrix (Gardner et al., 2018). However, these simple preconditioners may fail when the kernel matix is not low rank, which is the case for certain kernel length scales. HiGP adopts the newly proposed AFN preconditioner, which is designed for robust preconditioning of kernel matrices. Numerical experiments demonstrate that AFN can significantly improve the accuracy and robustness of iterative solvers for kernel matrix systems.

Lastly, HiGP uses accurate and efficient hand-coded gradient calculations. GPyTorch relies on automatic differentiation (autodiff) provided in PyTorch to calculate gradients (Formula (3)). Although autodiff is convenient and flexible, it is very inefficient when used to evaluate derivatives of quantitives that are computed iteratively, e.g., with PCG, due to long chain rule expressions resulting from the iterations.

Design and Implementation

We implemented HiGP in Python 3 and C++ with the goal of providing both a set of ready-to-use out-of-the-box Python interfaces for regular users and a set of reusable high-performance computational primitives for experienced users. The HiGP C++ part implements five functional units for performance-critical calculations:

1. The `kernel` unit. This unit populates the matrix block $\mathbf{K}(X, Y; l)$ and optionally $\partial\mathbf{K}(X, Y; l)/\partial l$ for two sets of points X, Y (typically subsets of the training or testing data), and a length scale l.
2. The `dense_kmat` unit. This unit computes the regularized kernel matrix block $\tilde{\mathbf{K}} = f^2\mathbf{K}(X, Y; l) + s\mathbf{I}$, and matrix products $\tilde{\mathbf{K}} \times B$ and $(\partial\tilde{\mathbf{K}}/\partial\theta) \times B$, where B is a general dense matrix and $\theta \in \{l, f, s\}$ is a hyperparameter.
3. The `h2mat` unit. This unit is similar to the `dense_kmat` unit, but only computes the $\mathcal{H}^2$ matrix-matrix multiplication for $\widehat{\mathbf{K}} \times B$ and $(\partial\widehat{\mathbf{K}}/\partial\theta) \times B$, where $\widehat{\mathbf{K}} = f^2\mathbf{K}(X, X; l) + s\mathbf{I}$ is a symmetric regularized kernel matrix.

4. The `solver` unit. This unit implements a PCG method for solving multiple right-hand-side systems simultaneously, employing the AFN preconditioner.
5. The `gp` unit. This unit implements the trace estimator and the computation of the loss and gradients in GP regression and GP classification computations. The loss and gradients can be computed in an "exact" manner using dense matrix factorization, or in a fast and approximate manner using preconditioned iterative solvers and stochastic Lanczos quadrature for trace estimation.

The aforementioned C++ units can be compiled as a standalone library with C language interfaces for secondary development in many programming languages, including C, C++, Python, Julia, and other languages.

HiGP wraps the C++ units into four basic Python modules:

1. `higp.krnlmatmodule` wraps and calls the C++ `dense_kmat` and `h2mat` units.
2. `higp.precondmodule` wraps and calls the PCG solver with the AFN precondioner; both are in the C++ `solver` unit.
3. `higp.gprproblemmodule` computes the loss and gradient for GP regression.
4. `higp.gpcproblemmodule` computes the loss and gradient for GP classification.

These basic Python modules provide fast access to the high-performance C++ units. Advanced users can utilize `higp.krnlmatmodule` and `higp.precondmodule` modules to develop new algorithms for kernel matrices. The Python interface allows faster and easier debugging and testing when prototyping new algorithms. The two modules `higp.gprproblemmodule` and `higp.gpcproblemmodule` allow a user to train a GP model with any gradient-based optimizer, allowing HiGP to be adopted in different data science and machine learning workflows.

To further simplify the training and use of GP models, we further implemented two high-level modules, `higp.GPRModel` and `higp.GPCModel`. These modules register the hyperparameters used in GP regression/classification as PyTorch parameters and set the gradients of PyTorch parameters in each step model the PyTorch optimizer. The Listing 1 below shows an example of defining and training a GP regression and using the trained model for prediction in just eight lines of code, where `pred.prediction_mean` and `pred.prediction_stddev` are the predicted mean values and the standard deviation of the predictive distribution for each data point in 'test_x.

```
# Listing 1: HiGP example code of training and using a GPR model
gprproblem = higp.gprproblem.setup(data=train_x, label=train_y, kernel_type=higp.Gaussia
model = higp.GPRModel(gprproblem)
optimizer = torch.optim.Adam(model.parameters(), lr=0.1)
for i in ranges(max_steps):
    loss = model.calc_loss_grad()
    optimizer.step()
params = model.get_params()
pred = higp.gpr_prediction(train_x, train_y, test_x, higp.GaussianKernel, params)
```

We note that the HiGP Python interfaces are *stateless*. For example, the same arguments `train_x`, `train_y`, and `higp.GaussianKernel` are passed into two functions `higp.gprproblem.setup` and `higp.gpr_prediction` in Listing 1. This design aims to simplify the interface and decouple different operations. A user can train and use different GP models with the same or different data and configurations in the same file.

Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., … Zheng, X. (2015). *TensorFlow: Large-scale machine learning on heterogeneous systems*. https://www.tensorflow.org/

Aune, E., Simpson, D. P., & Eidsvik, J. (2014). Parameter estimation in high dimensional Gaussian distributions. *Statistics and Computing*, *24*(2), 247–263. https://doi.org/10.

1007/s11222-012-9368-y

Bui, T., Hernandez-Lobato, D., Hernandez-Lobato, J., Li, Y., & Turner, R. (2016). Deep Gaussian processes for regression using approximate expectation propagation. In M. F. Balcan & K. Q. Weinberger (Eds.), *Proceedings of the 33rd international conference on machine learning* (Vol. 48, pp. 1472–1481). PMLR.

Burt, D. R., Artemev, A., & Wilk, M. van der. (2021). *Barely biased learning for Gaussian process regression*. https://arxiv.org/abs/2109.09417

Cai, D., Huang, H., Chow, E., & Xi, Y. (2023). Data-driven construction of hierarchical matrices with nested bases. *SIAM Journal on Scientific Computing*, *46*(2), S24–S50.

Chen, L., Chen, T., Detha, U., & Andersen, M. S. (2023). Towards scalable kernel-based regularized system identification. *2023 62nd IEEE Conference on Decision and Control (CDC)*, 1498–1504. https://doi.org/10.1109/CDC49753.2023.10384051

Damianou, A., & Lawrence, N. D. (2013). Deep Gaussian processes. In C. M. Carvalho & P. Ravikumar (Eds.), *Proceedings of the sixteenth international conference on artificial intelligence and statistics* (Vol. 31, pp. 207–215). PMLR.

Dutordoir, V., Salimbeni, H., Hambro, E., McLeod, J., Leibfried, F., Artemev, A., Wilk, M. van der, Hensman, J., Deisenroth, M. P., & John, S. (2021). *GPflux: A library for deep Gaussian processes*. https://arxiv.org/abs/2104.05674

Erlandson, L., Estrada Gómez, A. M., Chow, E., & Paynabar, K. (2025). smashGP: Large-scale spatial modeling via matrix-free Gaussian processes. *Journal of Computational and Graphical Statistics*, *34*(1), 15–33. https://doi.org/10.1080/10618600.2024.2353653

Gardner, J., Pleiss, G., Weinberger, K. Q., Bindel, D., & Wilson, A. G. (2018). GPyTorch: Blackbox matrix-matrix Gaussian process inference with GPU acceleration. *Advances in Neural Information Processing Systems*, *31*.

GPy. (since 2012). *GPy: A Gaussian process framework in python*. http://github.com/SheffieldML/GPy.

Hackbusch, W., & Börm, S. (2002). Data-sparse approximation by adaptive $\mathcal{H}^2$-matrices. *Computing*, *69*(1), 1–35.

Hackbusch, W., Khoromskij, B. N., & Sauter, S. A. (2000). On $\mathcal{H}^2$-matrices. In *Lectures on applied mathematics: Proceedings of the symposium organized by the sonderforschungsbereich 438 on the occasion of karl-heinz hoffmann's 60th birthday, munich, june 30 - july 1, 1999* (pp. 9–29). Max Planck Institute for Mathematics in the Sciences; Springer.

Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., … Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*(7825), 357–362. https://doi.org/10.1038/s41586-020-2649-2

Hensman, J., Fusi, N., & Lawrence, N. D. (2013). *Gaussian processes for big data*. 282–290. https://doi.org/10.5555/3023638.3023667

Huang, H., Xing, X., & Chow, E. (2020). H2Pack: High-performance h^2 matrix package for kernel matrices using the proxy point method. *ACM Transactions on Mathematical Software*, *47*(1), 1–29.

Hutchinson, M. F. (1989). A stochastic estimator of the trace of the influence matrix for laplacian smoothing splines. *Communications in Statistics - Simulation and Computation*, *18*(3), 1059–1076. https://doi.org/10.1080/03610918908812806

Jakkala, K. (2021). *Deep Gaussian processes: A survey*. https://arxiv.org/abs/2106.12135

Matthews, A. G. de G., van der Wilk, M., Nickson, T., Fujii, Keisuke., Boukouvalas, A.,

DRAFT

León-Villagrá, P., Ghahramani, Z., & Hensman, J. (2017). GPflow: A Gaussian process library using TensorFlow. *Journal of Machine Learning Research*, *18*(40), 1–6.

Meyer, R. A., Musco, C., Musco, C., & Woodruff, D. P. (2021). Hutch++: Optimal stochastic trace estimation. *Symposium on Simplicity in Algorithms (SOSA)*, 142–155.

Pleiss, G., Gardner, J., Weinberger, K., & Wilson, A. G. (2018). Constant-time predictive distributions for Gaussian processes. *Proceedings of the 35th International Conference on Machine Learning*, *80*, 4114–4123.

Potapczynski, A., Wu, L., Biderman, D., Pleiss, G., & Cunningham, J. P. (2021). Bias-free scalable Gaussian processes via randomized truncations. *Proceedings of the 38th International Conference on Machine Learning*, 8609–8619.

Rasmussen, C. E., & Williams, C. K. I. (2005). *Gaussian processes for machine learning*. The MIT Press. https://doi.org/10.7551/mitpress/3206.001.0001

Saad, Y. (2003). *Iterative methods for sparse linear systems*. Society for Industrial; Applied Mathematics. https://doi.org/10.1137/1.9780898718003

Titsias, M. (2009). Variational learning of inducing variables in sparse Gaussian processes. In D. van Dyk & M. Welling (Eds.), *Proceedings of the twelth international conference on artificial intelligence and statistics* (Vol. 5, pp. 567–574). PMLR.

Ubaru, S., Chen, J., & Saad, Y. (2017). Fast estimation of $tr(f(A))$ via stochastic lanczos quadrature. *SIAM Journal on Matrix Analysis and Applications*, *38*(4), 1075–1099. https://doi.org/10.1137/16M1104974

van der Wilk, M., Dutordoir, V., John, S., Artemev, A., Adam, V., & Hensman, J. (2020). *A framework for interdomain and multioutput Gaussian processes*. https://arxiv.org/abs/2003.01115

Wagner, T., Xu, T., Nestler, F., Xi, Y., & Stoll, M. (2025). *Preconditioned additive Gaussian processes with fourier acceleration*. https://arxiv.org/abs/2504.00480

Wenger, J., Pleiss, G., Hennig, P., Cunningham, J., & Gardner, J. (2022). Preconditioning for scalable Gaussian process hyperparameter optimization. *Proceedings of the 39th International Conference on Machine Learning*, 23751–23780.

Wilson, A. G., Dann, C., & Nickisch, H. (2015). *Thoughts on massively scalable Gaussian processes*.

Xing, X., & Chow, E. (2020). Interpolative decomposition via proxy points for kernel matrices. *SIAM Journal on Matrix Analysis and Applications*, *41*, 221–243.

Zhang, M., Chen, T., & Mu, B. (2024). Asymptotic properties of generalized maximum likelihood hyper-parameter estimator for regularized system identification. *2024 IEEE 63rd Conference on Decision and Control (CDC)*, 4961–4966. https://doi.org/10.1109/CDC56724.2024.10885816

Zhao, S., Xu, T., Huang, H., Chow, E., & Xi, Y. (2024). An adaptive factorized Nyström preconditioner for regularized kernel matrices. *SIAM Journal on Scientific Computing*, *46*(4), A2351–A2376. https://doi.org/10.1137/23M1565139

DRAFT